

PHONE ACCESSORIES

Catalog implementation tasks

DB + Backend + Angular

Единое ТЗ по задачам каталога

Source of truth по схеме: CATALOG_SCHEMA.md

Общие правила работы: AGENTS.md

Статусы задач ведутся в WEEEEK; этот документ хранит требования и порядок зависимостей.

1. Как использовать документ

- Документ объединяет все согласованные задачи по реализации catalog domain: 7 DB-задач, 8 backend-задач (включая Backend 4.1 Attribute Filtering) и 7 Angular-задач.
- Каждая задача выполняется только после своих зависимостей.
- CATALOG_SCHEMA.md определяет утвержденную target-схему данных. Код и миграции репозитория определяют, что уже фактически реализовано.
- AGENTS.md определяет общие ограничения: не ставить зависимости, не выполнять state-changing команды, миграции, тесты и другие запрещенные действия без явного подтверждения.
- Не добавлять сущности, поля, API, fallback-логику и UI-сценарии сверх соответствующего ТЗ без отдельного согласования.
- Если фактический код или API-контракт отличается от этого документа, не угадывать: сначала зафиксировать расхождение и согласовать решение.

2. Порядок выполнения

Работа организована конвейером DB -> Backend -> Angular. Когда backend текущего блока готов, frontend может выполняться параллельно со следующим DB-блоком.

1. Alex: Catalog DB - 1. Category + Brand
2. Alex: Catalog Backend - 1. Category + Brand
3. ПАРАЛЛЕЛЬНО: Илья - Catalog Angular - 1. Category + Brand; Alex - Catalog DB - 2. Product + ProductCategory + ProductVariant
4. Alex: Catalog Backend - 2. Product + ProductCategory + ProductVariant
5. ПАРАЛЛЕЛЬНО: Илья - Catalog Angular - 2. Product + ProductCategory + ProductVariant; Alex - Catalog DB - 3. Attribute + AttributeOption + CategoryAttribute
6. Alex: Catalog Backend - 3. Attribute + AttributeOption + CategoryAttribute
7. Alex: Catalog DB - 4. ProductAttributeValue + VariantAttributeValue
8. Alex: Catalog Backend - 4. ProductAttributeValue + VariantAttributeValue
9. ПАРАЛЛЕЛЬНО: Илья - Catalog Angular - 4. ProductAttributeValue + VariantAttributeValue; Alex - Catalog Backend - 4.1. Attribute Filtering
10. Илья: Catalog Angular - 3. Attribute Filters
11. Alex: Catalog DB - 5. ProductMedia
12. Alex: Catalog Backend - 5. ProductMedia
13. ПАРАЛЛЕЛЬНО: Илья - Catalog Angular - 5. ProductMedia; Alex - Catalog DB - 6. Stock
14. Alex: Catalog Backend - 6. Stock
15. ПАРАЛЛЕЛЬНО: Илья - Catalog Angular - 6. Stock; Alex - Catalog DB - 7. ProductCompatibility
16. Alex: Catalog Backend - 7. ProductCompatibility
17. Илья: Catalog Angular - 7. ProductCompatibility

Важно: Angular 3 идет после Backend 4.1, а Angular 4 может выполняться сразу после Backend 4. Это намеренно: Backend 3 описывает, какие характеристики можно фильтровать, а реальные значения для фильтрации появляются после DB/Backend 4.

3. Catalog DB tasks

Catalog DB - 1. Category + Brand

Цель: реализовать первый блок catalog domain: Category и Brand.

Перед началом

- Прочитать AGENTS.md и CATALOG_SCHEMA.md.
- Проверить текущее состояние backend/catalog/models.py и существующих migrations.
- Не менять accounts и frontend.

Category

- id - стандартный PK Django.
- parent - self ForeignKey, nullable/blank, on_delete=PROTECT, related_name="children".
- name - CharField(max_length=255), required.
- slug - SlugField(max_length=255), required, unique.
- description - TextField(blank=True).
- is_active - BooleanField(default=True).
- sort_order - PositiveIntegerField(default=0).
- created_at - DateTimeField(auto_now_add=True).
- updated_at - DateTimeField(auto_now=True).
- Root category имеет parent=NULL.
- Дерево реализовать обычным adjacency list через parent.
- Не подключать MPTT/tree-библиотеки.
- Удаление родительской категории с дочерними должно блокироваться через PROTECT.
- Добавить нормальный __str__.

Brand

- id - стандартный PK Django.
- name - CharField(max_length=255), required, unique.
- slug - SlugField(max_length=255), required, unique.
- description - TextField(blank=True).
- is_active - BooleanField(default=True).
- created_at - DateTimeField(auto_now_add=True).
- updated_at - DateTimeField(auto_now=True).
- Добавить нормальный __str__.

Не добавлять

- Brand.logo, website, country, featured, sort_order.
- Прямую связь Brand <-> Category.

Команды и проверка

Создать новую migration для catalog. Migration создавать от host UID/GID:

```
docker compose exec --user "$(id -u):$(id -g)" backend python manage.py makemigrations catalog
docker compose exec backend python manage.py check
```

migrate пока не запускать.

Результат задачи

- Измененные файлы.
- git diff.
- Содержимое созданной migration.
- Результат manage.py check.

Catalog DB - 2. Product + ProductCategory + ProductVariant

Зависимость: Catalog DB - 1 завершена.

Цель: реализовать Product, ProductCategory и ProductVariant по CATALOG_SCHEMA.md.

Product

- brand - FK -> Brand, null=True, blank=True, on_delete=PROTECT.
- name - CharField(max_length=255).
- slug - SlugField(max_length=255, unique=True).
- description - TextField(blank=True).
- status - CharField с TextChoices: draft, active, archived.
- created_at, updated_at.
- Product не должен содержать price, stock, image, color/storage, processor/battery и другие характеристики.

ProductCategory

- product - FK -> Product, CASCADE.
- category - FK -> Category, PROTECT.
- is_primary - BooleanField(default=False).
- Constraint UNIQUE(product, category).
- Максимум одна запись is_primary=True на Product через conditional UniqueConstraint.
- Не требовать primary category на DB-уровне: draft Product может быть временно неполным.

ProductVariant

- product - FK -> Product, CASCADE.
- sku - CharField(max_length=128), required, unique.
- barcode - CharField(max_length=64), nullable/blank, unique when present.
- price - DecimalField(max_digits=12, decimal_places=2).
- compare_at_price - DecimalField(max_digits=12, decimal_places=2, nullable/blank).
- is_active - BooleanField(default=True).
- is_default - BooleanField(default=False).
- sort_order - PositiveIntegerField(default=0).
- created_at, updated_at.
- Constraint price >= 0.
- Constraint compare_at_price IS NULL OR compare_at_price >= price.
- Максимум один is_default=True на Product.
- Отсутствующий barcode хранить как NULL.

Не добавлять

- currency.
- stock quantity в ProductVariant.
- fixed color/storage/size/material.
- weight/dimensions.
- отдельную цену в Product.

Даже товар без видимых опций позже должен иметь ProductVariant.

Проверка

- Создать новую migration, не изменяя предыдущую.
- makemigrations catalog через host UID/GID.
- manage.py check.
- migrate не запускать.

Результат задачи

- git diff.

- Migration.
- Результат manage.py check.

Catalog DB - 3. Attribute + AttributeOption + CategoryAttribute

Зависимость: Catalog DB - 1-2 завершены.

Цель: реализовать метаданные системы характеристик.

Attribute

- name - CharField(max_length=255).
- slug - SlugField(max_length=255, unique=True).
- data_type - TextChoices: text, integer, decimal, boolean, option.
- unit - CharField(max_length=64, blank=True).
- is_active - BooleanField(default=True).
- created_at, updated_at.
- Не добавлять глобальные is_filterable и is_variant_attribute.

AttributeOption

- attribute - FK -> Attribute.
- value - CharField(max_length=255).
- is_active - BooleanField(default=True).
- sort_order - PositiveIntegerField(default=0).
- created_at, updated_at.
- Constraint UNIQUE(attribute, value).
- При удалении неиспользуемого Attribute его options могут удаляться вместе с ним.
- Если AttributeOption уже используется значениями товаров, удаление позже должно блокироваться через PROTECT со стороны value-моделей.

CategoryAttribute

- category - FK -> Category.
- attribute - FK -> Attribute.
- scope - TextChoices: product, variant.
- is_required - BooleanField(default=False).
- is_filterable - BooleanField(default=False).
- sort_order - PositiveIntegerField(default=0).
- Constraint UNIQUE(category, attribute).
- Удаление Category может удалить ее CategoryAttribute.
- Attribute, используемый CategoryAttribute, должен быть защищен через PROTECT.

Не реализовывать

- Inheritance характеристик от parent Category.
- AttributeGroup / AttributeSet.
- multiselect.
- min/max, regex/default values.
- conditional attributes.

Проверка

- Создать новую migration.
- makemigrations catalog через host UID/GID.
- manage.py check.
- migrate не запускать.

Результат задачи

- git diff.

- Migration.
- Результат manage.py check.

Catalog DB - 4. ProductAttributeValue + VariantAttributeValue

Зависимость: Catalog DB - 1-3 завершены.

Цель: реализовать typed storage значений характеристик.

ProductAttributeValue

- product - FK -> Product, CASCADE.
- attribute - FK -> Attribute, PROTECT.
- value_text - nullable.
- value_integer - nullable.
- value_decimal - nullable.
- value_boolean - nullable.
- option - FK -> AttributeOption, nullable/blank, PROTECT.
- Constraint UNIQUE(product, attribute).
- Ровно одно из value_text/value_integer/value_decimal/value_boolean/option должно быть NOT NULL.

VariantAttributeValue

- variant - FK -> ProductVariant, CASCADE.
- attribute - FK -> Attribute, PROTECT.
- Те же typed value fields.
- option - FK -> AttributeOption, nullable/blank, PROTECT.
- Constraint UNIQUE(variant, attribute).
- Ровно одно typed value поле должно быть NOT NULL.

Правила

- Для decimal использовать DecimalField с достаточной точностью для характеристик.
- Числовые характеристики хранить числами, boolean - boolean.
- Не использовать JSONField для основных характеристик.
- Не использовать одно универсальное string value.
- Не создавать Phone/Laptop/Case value tables.
- На этом этапе реализовать DB-level constraints.

Не добавлять пока

- Отдельную API/admin/service-валидацию соответствия typed field -> Attribute.data_type.
- Проверку принадлежности option -> тому же Attribute.
- Эти cross-table invariants реализуются в Backend - 4.

Проверка

- Создать новую migration.
- makemigrations catalog через host UID/GID.
- manage.py check.
- migrate не запускать.

Результат задачи

- git diff.
- Migration.
- Результат manage.py check.

Catalog DB - 5. ProductMedia

Зависимость: Product и ProductVariant реализованы.

Цель: реализовать ProductMedia по CATALOG_SCHEMA.md.

Перед началом

- Проверить backend/requirements.txt.
- Для Django ImageField требуется Pillow.
- Если Pillow отсутствует - не устанавливать и не менять dependencies самостоятельно; остановиться и сообщить Илье.
- Dependency change выполняется только после отдельного согласования.

ProductMedia

- product - FK -> Product, CASCADE.
- variant - FK -> ProductVariant, null=True, blank=True, CASCADE.
- file - storage-backed ImageField.
- alt_text - CharField(max_length=255, blank=True).
- sort_order - PositiveIntegerField(default=0).
- is_primary - BooleanField(default=False).
- created_at, updated_at.
- Максимум одно общее primary media на Product: is_primary=True AND variant IS NULL.
- Максимум одно primary media на Variant.

Правила

- Не хранить binary в PostgreSQL.
- Не использовать direct filesystem paths в domain code.
- Invariant: если variant указан, variant.product должен совпадать с product.
- Cross-table invariant не пытаться изображать неправильным DB CHECK.

Не добавлять

- Google Drive URL.
- external_url fallback.
- storage_provider.
- video.
- thumbnail subsystem.
- width/height/file_size.
- production S3 config.

Проверка

- Создать новую migration только после решения вопроса ImageField/Pillow.
- manage.py check.
- migrate не запускать.

Результат задачи

- git diff.
- Migration.
- Результат manage.py check.
- Отдельно указать, потребовались ли dependency changes.

Catalog DB - 6. Stock

Зависимость: ProductVariant реализован.

Цель: реализовать Stock как отдельную inventory-модель.

Stock

- variant - OneToOneField или FK unique -> ProductVariant, CASCADE.
- quantity - PositiveIntegerField(default=0).

- reserved - PositiveIntegerField(default=0).
- updated_at - DateTimeField(auto_now=True).
- Constraint quantity >= 0.
- Constraint reserved >= 0.
- Constraint reserved <= quantity.
- Одна Stock запись на один ProductVariant.

Правила

- available = quantity - reserved; available не хранить в БД.
- Не добавлять is_in_stock.
- reserved нужен для будущего checkout, но механику reservation сейчас не реализовывать.
- Stock относится только к ProductVariant, не к Product.

Не добавлять

- Warehouse / WarehouseStock.
- InventoryMovement / StockHistory.
- SupplierStock.
- reorder level.
- backorders.

Проверка

- Создать новую migration.
- makemigrations catalog через host UID/GID.
- manage.py check.
- migrate не запускать.

Результат задачи

- git diff.
- Migration.
- Результат manage.py check.

Catalog DB - 7. ProductCompatibility

Зависимость: Product реализован.

Цель: реализовать направленную связь совместимости Product -> Product.

ProductCompatibility

- product - FK -> Product, CASCADE.
- compatible_product - FK -> Product, CASCADE.
- created_at - DateTimeField(auto_now_add=True).
- Для двух FK на Product использовать разные понятные related_name.
- Constraint UNIQUE(product, compatible_product).
- Constraint product != compatible_product.

Семантика

Связь направленная.

```
Silicone Case -> compatible with -> iPhone 16
```

Это не создает автоматическую обратную запись.

Не добавлять

- free-form compatible_models string.
- VariantCompatibility.
- CompatibilityType / CompatibilityGroup.

- compatibility rules engine.

Проверка

- Создать новую migration.
- makemigrations catalog через host UID/GID.
- manage.py check.
- migrate не запускать.

Результат задачи

- git diff.
- Migration.
- Результат manage.py check.

4. Catalog Backend tasks

Общий принцип backend-линейки: Django Admin используется для внутреннего управления каталогом, публичный DRF API для Angular - read-only, если конкретная задача явно не утверждает иное.

Catalog Backend - 1. Category + Brand

Зависимость: Catalog DB - 1 завершена, migration применена.

Цель: добавить внутреннее управление Category/Brand через Django Admin и публичный read-only API для Angular.

Общие правила

- API держать под /api/catalog/.
- Существующий /api/health/ сохранить без изменений.
- Не менять frontend.
- Не устанавливать новые dependencies.

Django Admin

- Category: list_display name, parent, slug, is_active, sort_order.
- Category: search_fields name, slug.
- Category: list_filter is_active.
- Category: prepopulated_fields для slug из name.
- Category: ordering sort_order, name.
- Brand: list_display name, slug, is_active.
- Brand: search_fields name, slug.
- Brand: list_filter is_active.
- Brand: prepopulated_fields для slug из name.
- Brand: ordering name.
- Не делать custom admin forms без необходимости.

Public API

```
GET /api/catalog/categories/  
GET /api/catalog/categories/<slug>/  
  
GET /api/catalog/brands/  
GET /api/catalog/brands/<slug>/
```

- Только read-only. Не добавлять POST/PUT/PATCH/DELETE.
- Detail lookup по slug, не id.
- Category API: id, parent_id, name, slug, description, sort_order.
- Публично отдавать только is_active=True Category.
- Category ordering: sort_order, name.
- Не строить recursive children tree на backend; frontend получает parent_id.
- Brand API: id, name, slug, description.
- Публично отдавать только is_active=True Brand.
- Brand ordering: name.

Структура

- Добавить serializers.py.
- Использовать стандартные DRF serializers/views/viewsets.
- Не создавать services/selectors для простой read-only CRUD-логики.
- DRF router допустим; routes остаются app-level в catalog/urls.py.

Проверка

```
docker compose exec backend python manage.py check

curl http://127.0.0.1:8000/api/catalog/categories/
curl http://127.0.0.1:8000/api/catalog/brands/
curl http://127.0.0.1:4200/api/catalog/categories/
curl http://127.0.0.1:4200/api/catalog/brands/
```

Тесты не создавать и не запускать.

Результат задачи

- Измененные файлы.
- git diff.
- manage.py check.
- curl.
- Что не проверялось.

Catalog Backend - 2. Product + ProductCategory + ProductVariant

Зависимость: Catalog DB - 2 и Catalog Backend - 1 завершены.

Цель: добавить управление товарами через Django Admin и публичный API каталога.

Django Admin

- Product list_display: name, brand, status, slug, created_at.
- Product search_fields: name, slug.
- Product list_filter: status, brand.
- Product prepopulated_fields для slug из name.
- ProductCategory inline в Product admin.
- ProductVariant inline в Product admin.
- ProductVariant можно зарегистрировать отдельно: list_display sku, product, price, is_active, is_default.
- ProductVariant search_fields: sku, barcode, product__name.
- ProductVariant list_filter: is_active, is_default.
- Не создавать Angular admin panel.

Public API

```
GET /api/catalog/products/
GET /api/catalog/products/<slug>/
```

- Только read-only.
- Публично отдавать только Product.status=active.
- Variants в public API только is_active=True.
- Product list: id, name, slug, brand|null, default_variant|null.
- brand: id, name, slug.
- default_variant: id, sku, price, compare_at_price.
- Если is_default variant отсутствует, default_variant=null; не выбирать первый variant fallback-ом.
- Product detail дополнительно: description, categories, variants.
- Category item: id, name, slug, is_primary.
- Variant: id, sku, barcode, price, compare_at_price, is_default, sort_order.
- Variants ordering: sort_order, id.

Query params

```
?category=<category-slug>
?brand=<brand-slug>
```

- Category filter работает по прямой ProductCategory связи.
- Не искать автоматически по дочерним категориям.
- Не добавлять пока search, pagination, price sorting, availability, attributes, media.

Query efficiency

- Не создавать очевидный N+1.
- Использовать select_related/prefetch_related там, где это требуется текущими serializers.
- Не вводить selectors/services только ради query.

Проверка

```
docker compose exec backend python manage.py check

GET /api/catalog/products/
GET /api/catalog/products/?brand=<slug>
GET /api/catalog/products/?category=<slug>
GET /api/catalog/products/<slug>/
```

Тесты не создавать.

Результат задачи

- Измененные файлы.
- git diff.
- manage.py check.
- curl.
- Что не проверено.

Catalog Backend - 3. Attribute + AttributeOption + CategoryAttribute

Зависимость: Catalog DB - 3 и предыдущие backend-задачи завершены.

Цель: управлять системой характеристик через Django Admin и отдавать Angular metadata категории.

Django Admin

- Attribute list_display: name, slug, data_type, unit, is_active.
- Attribute search_fields: name, slug.
- Attribute list_filter: data_type, is_active.
- Attribute prepopulated_fields для slug из name.
- AttributeOption inline внутри Attribute admin: value, is_active, sort_order.
- CategoryAttribute inline в существующий Category admin: attribute, scope, is_required, is_filterable, sort_order.
- Отдельная CategoryAttribute admin-модель допустима для просмотра связей.

Public API

Не добавлять отдельные публичные endpoints /attributes/, /attribute-options/, /category-attributes/.
Расширить Category detail.

```
GET /api/catalog/categories/<slug>/

attributes: [
  {
    id,
    name,
    slug,
    data_type,
    unit,
    scope,
    is_required,
```

```

    is_filterable,
    sort_order,
    options: [...]
}
]

```

- options заполнять только для data_type=option.
- Option: id, value, sort_order.
- Публично отдавать только Attribute.is_active=True и AttributeOption.is_active=True.
- CategoryAttribute ordering: sort_order, attribute name.
- Category list не раздувать attributes; metadata нужна только в detail.

Не реализовывать

- Inheritance от parent category.
- Global is_filterable / is_variant_attribute.
- AttributeGroup.
- multiselect.
- dynamic filter query engine.
- product attribute values.

Проверка

- manage.py check.
- Проверить category detail без attributes.
- Проверить category detail с attributes, если данные заведены.
- Тесты не создавать.

Результат задачи

- git diff.
- manage.py check.
- curl.
- Что не проверено.

Catalog Backend - 4. ProductAttributeValue + VariantAttributeValue

Зависимость: Catalog DB - 4 и Catalog Backend - 1-3 завершены.

Цель: реализовать application-level validation typed values, admin и выдачу характеристик в Product API.

Validation

- data_type=text -> value_text.
- integer -> value_integer.
- decimal -> value_decimal.
- boolean -> value_boolean.
- option -> option.
- Если выбран option, option.attribute должен совпадать с attribute текущего value.
- Validation должна работать при сохранении через Django Admin и выдавать normal validation error.
- Не дублировать проверку в нескольких несвязанных местах; использовать простое централизованное решение.
- Не реализовывать required CategoryAttribute publication validation, конфликты нескольких категорий и write API.

Django Admin

- ProductAttributeValue inline в Product admin.
- VariantAttributeValue inline в ProductVariant admin.

- Typed value fields оставить явными.
- Не делать сложный dynamic JS admin.

Public Product API

Расширить GET /api/catalog/products/<slug>/.

```
attributes: [
  {
    attribute: {
      id,
      name,
      slug,
      data_type,
      unit
    },
    value: ...
  }
]
```

- text -> string.
- integer -> integer.
- decimal -> string с точным decimal value.
- boolean -> boolean.
- option -> { id, value }.
- Не отдавать пять внутренних typed columns.
- Для каждого variant добавить attributes: [...].
- Product-level и variant-level attributes не смешивать.
- Ordering брать из CategoryAttribute.sort_order там, где есть однозначная настройка.
- Если при нескольких категориях порядок неоднозначен - сначала сообщить Илье, не добавлять fallback наугад.

Не добавлять

- write API.
- JSON storage.
- характеристики строкой вида '5000 mAh'.
- publication workflow.
- search/filter engine.

Проверка

- manage.py check.
- Через Admin: корректный typed value сохраняется.
- Неправильный type блокируется validation.
- Option другого Attribute блокируется.
- Product detail проверить через curl.
- Тесты не создавать.

Результат задачи

- git diff.
- manage.py check.
- Результаты ручной/API проверки.

Catalog Backend - 4.1. Attribute Filtering

Зависимость: Catalog Backend - 3 и Catalog Backend - 4 завершены.

Цель: реализовать server-side filtering GET /api/catalog/products/ по CategoryAttribute с is_filterable=True.

Ограничения

- Не устанавливать django-filter и другие dependencies без отдельного согласования.
- ProductAttributeValue и VariantAttributeValue уже должны быть доступны.

API contract

```
GET /api/catalog/products/?category=<slug>&attr[<attribute-slug>]=<value>
```

Примеры:

```
?category=smartphones&attr[color]=12  
?category=smartphones&attr[storage]=25  
?category=cases&attr[magsafe]=true
```

- Для data_type=option значение query param - AttributeOption.id.
- Для text/integer/decimal/boolean использовать typed exact value по data_type.
- Поддержать повтор одного attribute: ?attr[color]=12&attr[color]=13.
- Несколько значений одного Attribute = OR.
- Разные Attributes = AND.
- Product-scope attributes фильтровать через ProductAttributeValue.
- Variant-scope attributes фильтровать через VariantAttributeValue.

Критичный invariant для variants

Если указано несколько variant-scope filters, они должны совпасть на одном и том же ProductVariant. Нельзя считать Product подходящим, если Color=Black находится на одном SKU, а Storage=256 на другом SKU, когда фильтр требует Black + 256.

Public catalog rules

- Учитывать только Product.status=active.
- Учитывать только ProductVariant.is_active=True для variant-scope filtering.
- Не допускать duplicate Products в queryset.

Не реализовывать

- client-side filtering.
- ranges/min/max.
- fuzzy matching.
- full-text search.
- category inheritance.
- multiselect отдельной моделью.
- новые dependencies.

Проверка

- manage.py check.
- curl нескольких комбинаций.
- OR внутри одного Attribute.
- AND разных Attributes.
- Отдельно несколько variant attributes на одном SKU.
- Тесты без отдельного требования не создавать.

Результат задачи

- Измененные файлы.
- git diff.
- manage.py check.
- curl.
- Что не проверялось.

Catalog Backend - 5. ProductMedia

Зависимость: Catalog DB - 5 завершена; вопрос ImageField/Pillow отдельно согласован.

Цель: управлять изображениями через Django Admin и отдавать media в Product API.

Dependency safety

- Не выполнять pip install и не менять dependencies самостоятельно.
- Если Pillow отсутствует или требуется новая dependency/configuration - остановиться и сообщить Илье.

Validation

- Если ProductMedia.variant указан, variant.product_id должен совпадать с ProductMedia.product_id.
- Admin должен выдавать validation error, а не позволять сохранить media от Variant другого Product.
- DB constraints primary media остаются источником истины для uniqueness.

Django Admin

- Зарегистрировать ProductMedia.
- Поля: product, variant, file, alt_text, sort_order, is_primary.
- Добавить ProductMedia inline в Product admin, если не требуется сложная custom admin-логика.
- Не создавать image processing subsystem.

Development storage

- Если local media configuration отсутствует, добавить минимальную стандартную Django-конфигурацию через storage abstraction.
- MEDIA_URL.
- MEDIA_ROOT.
- Dev serving media через Django только при DEBUG.
- Не использовать direct filesystem paths в domain code.
- Не добавлять S3, boto, Google Drive, external URL fallback, thumbnails, compression/resizing, production media config.

Public Product API

- Product list: добавить primary_image.
- primary_image брать только из general ProductMedia: variant IS NULL и is_primary=True.
- Если отсутствует - primary_image=null. Не fallback на variant image.
- primary_image format: { url, alt_text }.
- Product detail: general product media [{id, url, alt_text, sort_order, is_primary}].
- В каждый variant добавить собственный media массив.
- Media ordering: sort_order, id.

Проверка

- manage.py check.
- Admin: загрузить Product image.
- Admin: загрузить Variant image.
- Проверить validation неправильного product/variant.
- Через API проверить URL изображения через backend и Angular proxy.
- Тесты не создавать.

Результат задачи

- git diff.
- Согласованные dependency/config changes.

- manage.py check.
- curl.
- Что не проверено.

Catalog Backend - 6. Stock

Зависимость: Catalog DB - 6 завершена.

Цель: управлять остатками через Django Admin и отдавать Angular состояние наличия SKU.

Django Admin

- Stock inline в ProductVariant admin.
- Показывать quantity, reserved, вычисляемое available=quantity-reserved.
- Если Stock зарегистрирован отдельно: list_display variant, quantity, reserved, available.
- Search по variant SKU/product name.
- available не сохранять в БД.

Validation

- DB constraints: quantity ≥ 0 , reserved ≥ 0 , reserved $\leq$ quantity.
- Для Admin добавить normal validation error при reserved $>$ quantity.
- Не добавлять reservation service.
- Не изменять stock автоматически.

Public API

- Не создавать /stocks/.
- Не отдавать exact quantity/reserved покупателю.
- Расширить ProductVariant полем in_stock:boolean.
- in_stock=true, если Stock существует и quantity-reserved > 0 .
- Если Stock отсутствует, in_stock=false - это явный API-контракт.
- Добавить in_stock в default_variant Product list serializer.

Не добавлять

- available_quantity в public API.
- is_in_stock в DB.
- warehouses.
- reservation behavior.
- stock history.

Query efficiency

- Не создавать N+1 при получении stock для variants/default_variant.

Проверка

- manage.py check.
- stock 10 / reserved 2 -> in_stock=true.
- stock 0 -> false.
- reserved == quantity -> false.
- reserved $>$ quantity блокируется validation.
- Product list/detail проверить curl.
- Тесты не создавать.

Результат задачи

- git diff.
- Результаты проверок.

Catalog Backend - 7. ProductCompatibility

Зависимость: Catalog DB - 7 завершена.

Цель: управлять совместимостью в Django Admin и отдавать ее в Product API.

Django Admin

- ProductCompatibility inline в Product admin по product FK.
- Поле compatible_product.
- Отдельная ProductCompatibility admin-модель допустима: product, compatible_product, created_at.
- Не делать автоматическую обратную запись.
- Self-compatibility должна давать normal validation error.
- DB constraint остается обязательным.

Public API

Не создавать сырой /product-compatibilities/. Расширить Product detail.

```
compatible_with: [...]  
compatible_items: [...]
```

- compatible_with: записи, где ProductCompatibility.product=current_product.
- compatible_items: обратная сторона, где compatible_product=current_product.
- Элемент списка: id, name, slug, brand|null, primary_image если media backend уже реализован.
- Отдавать только Product.status=active.

Не добавлять

- VariantCompatibility.
- CompatibilityType.
- automatic bidirectional rows.
- fuzzy/model-name matching.
- free-form compatibility strings.

Query efficiency

- Использовать prefetch/select_related, чтобы обе коллекции не создавали N+1.

Проверка

Создать через Admin пример Silicone Case -> iPhone 16.

- GET Case: compatible_with содержит iPhone 16.
- GET iPhone 16: compatible_items содержит Silicone Case.
- Self-compatibility блокируется.
- manage.py check.
- Тесты не создавать.

Результат задачи

- git diff.
- manage.py check.
- curl.
- Что не проверялось.

5. Catalog Angular tasks

Frontend работает только с реальными backend contracts. Не использовать mock data, скрытые fallback-и и client-side бизнес-логику вместо backend.

Catalog Angular - 1. Category + Brand

Зависимость: Catalog Backend - 1 завершена, API проверен.

Цель: создать основу frontend-каталога и вывести реальные Category/Brand.

Перед началом

- Прочитать AGENTS.md.
- Проверить реальный API-контракт Backend - 1.
- Не устанавливать dependencies.
- Не использовать mock data.
- API-вызовы только через относительные /api/... paths.

Структура

```
web/src/app/features/catalog/  
├── data-access/  
├── pages/  
└── ui/
```

- Не создавать отдельные feature для каждой DB-модели.
- Подключить HttpClient штатным Angular способом в app.config.ts.
- Standalone components.
- ChangeDetectionStrategy.OnPush.
- signals для local state, где уместно.
- @if / @for.
- strict TypeScript/templates.
- Tailwind CSS.
- Минимум component CSS.

API и types

```
GET /api/catalog/categories/  
GET /api/catalog/categories/<slug>/  
GET /api/catalog/brands/  
GET /api/catalog/brands/<slug>/
```

Создать frontend-типы строго под реальные responses и Catalog API/data-access service.

Routing

```
/catalog  
/catalog/categories/:slug  
/catalog/brands/:slug
```

- Root App использует RouterOutlet.
- Angular starter screen заменить только в объеме, необходимом для маршрутизации каталога.

Catalog page

- Вывести список активных категорий и брендов.
- Category: name, description если есть.
- Brand: name, description если есть.
- Категории отображать в порядке API.
- Можно построить визуальную иерархию через parent_id из уже полученных данных.
- Не делать дополнительных запросов для каждого элемента.

Detail pages

- Category detail: name, description.
- Brand detail: name, description.
- Пока Product API нет - не рисовать fake products и counters.

States

- loading.
- empty state.
- API error state.
- Не добавлять mock/fallback data при ошибке.

Не делать

- product catalog.
- filters.
- cart.
- auth.
- admin UI.
- UI libraries.
- design system.
- сложную global layout систему.

Проверка

- /catalog.
- Category detail.
- Brand detail.
- Прямое открытие route.
- Загрузка данных через /api/.
- Отсутствие hardcoded backend URL.
- Тесты и npm/ng build без отдельной команды не запускать.

Результат задачи

- Измененные файлы.
- Что реализовано.
- Что проверено.
- Что не проверено.

Catalog Angular - 2. Product + ProductCategory + ProductVariant

Зависимость: Catalog Backend - 2 и Catalog Angular - 1 завершены.

Цель: реализовать основной каталог товаров и страницу товара на реальном Product API.

Data access

- Добавить frontend types для Product list/detail, ProductVariant, ProductCategory, nested Brand.
- Поддержать GET /api/catalog/products/ и GET /api/catalog/products/<slug>/.
- Поддержать query params category и brand.
- Не менять backend contract на frontend.

Catalog page

- Расширить /catalog и вывести Product list.
- Product card: name, brand, цена default_variant, compare_at_price, ссылка на detail.
- Пока media нет - не использовать mock images/placeholder URL.
- Если default_variant=null, не выбирать первый variant fallback-ом.

Category/Brand navigation

```
/catalog?category=<slug>  
/catalog?brand=<slug>  
/catalog?category=<slug>&brand=<slug>
```

- URL является источником состояния Category/Brand filters.
- При query params changes повторно получать Product list с соответствующими params.

Product detail

```
/catalog/products/:slug
```

- Показать name, brand, description, categories, variants.
- Variant: SKU, price, compare_at_price.
- Если API содержит is_default=true - выбрать этот variant.
- Если default отсутствует - не выбирать первый автоматически.
- Selected variant хранить local signal.
- При выборе variant обновлять отображаемую цену.
- Stock пока не придумывать.

UI components

- По необходимости ProductCard.
- VariantSelector.
- Category/Brand filters.
- Не создавать abstraction ради одного использования.

Не делать

- attribute filters.
- product specifications.
- images/gallery.
- stock.
- compatibility.
- cart/add-to-cart.
- pagination, если backend ее не предоставляет.
- client-side fake filtering.

Проверка

- /catalog.
- Category filter.
- Brand filter.
- Оба filters вместе.
- Direct URL with query params.
- Product detail.
- Variant switching.
- Product без brand/default variant, если такие реальные данные есть.
- Тесты/build без отдельной команды не запускать.

Catalog Angular - 3. Attribute Filters

Зависимость: Catalog Backend - 3 дает metadata; Catalog Backend - 4.1 дает реальную server-side attribute filtering.

Цель: построить динамические фильтры каталога на основе CategoryAttribute metadata.

Важно

- Одного Backend - 3 недостаточно для рабочего filtering.
- Если backend filter contract еще не готов - задача blocked.

- Не реализовывать неработающие UI filters.
- Не фильтровать Product локально.
- Не загружать все Product detail ради filtering.

Metadata

- При выбранной Category получать GET /api/catalog/categories/<slug>/.
- Использовать attributes из API.
- Показывать только is_filterable=true.
- Учитывать slug, data_type, unit, options, sort_order.

Filter UI

- Фильтры строить динамически, не хардкодить Color/Storage/RAM/Power.
- option -> варианты из API.
- boolean -> boolean-control.
- integer/decimal -> не придумывать min/max/range, если backend contract их не предоставляет.
- Использовать только согласованный query format.

URL state

- Attribute filters хранить в query params по утвержденному backend contract.
- Не придумывать свой формат.
- Reload/deep link должен восстанавливать UI из URL.

Product loading

- Выбранные filters отправлять backend в GET /api/catalog/products/.
- Не выполнять business filtering в Angular.

Category change

- Загрузить metadata новой Category.
- Удалить из URL filters, которые к новой Category не относятся.
- Не переносить несовместимые attribute values.

Не делать

- hardcoded filters.
- client-side filtering.
- category inheritance.
- multiselect если backend его не поддерживает.
- min/max guessing.
- дополнительную frontend schema.

Проверка

- Минимум одну Category с option-filter.
- Минимум один другой поддерживаемый data_type.
- URL state.
- Reload.
- Category change.
- Реальные backend filtering results.
- Empty result.
- Если backend filtering не готов - задача blocked, не частично выполнена.

Catalog Angular - 4. ProductAttributeValue + VariantAttributeValue

Зависимость: Catalog Backend - 4 и Catalog Angular - 2 завершены.

Цель: показать реальные характеристики Product и выбранного ProductVariant.

Types

- Расширить Product detail types по реальному API.
- Attribute: id, name, slug, data_type, unit.
- Value: string/number/boolean или AttributeOption object.
- Не использовать внутренние DB поля value_text/value_integer/value_decimal/value_boolean/option_id.

Product specifications

- На Product detail добавить блок 'Характеристики'.
- Показывать Product.attributes.
- Формат: Attribute name | displayed value.
- Если unit есть - добавлять при отображении: 5000 mAh, 30 W, 6.3 inch.
- Boolean: Да / Нет.
- Option: option.value.
- Не строить отдельную разметку под каждый Attribute slug.

Variant attributes

- При выборе variant показывать его attributes.
- При переключении variant блок обновляется без reload.
- Product-level attributes остаются неизменными.

Empty states

- Если Product.attributes пуст - не показывать пустой блок.
- Если Variant.attributes пуст - не показывать пустой variant block.
- Не подменять отсутствие данных mock values.

Не делать

- attribute editing.
- admin UI.
- client-side DB type validation.
- JSON characteristics.
- filter engine в этой задаче.
- hardcoded Processor/Battery components.

Проверка

- Проверить существующие text/integer/decimal/boolean/option значения.
- Если какого-то типа нет в реальных данных - явно указать, что он не проверялся.
- Проверить смену variant.
- Тесты/build без отдельной команды не запускать.

Catalog Angular - 5. ProductMedia

Зависимость: Catalog Backend - 5 завершена, media URLs доступны; Catalog Angular - 2 завершена.

Цель: добавить изображения товаров и variants в каталог.

Types

- Product summary: primary_image|null.
- Product detail: media[[]].
- ProductVariant: media[[]].
- Использовать URL backend; не строить media path самостоятельно.

Product cards

- Если primary_image != null - показать image с alt_text из API.

- Если null - не использовать внешний placeholder URL.
- Показать нейтральное empty media state собственным UI.

Product gallery

- Простая gallery: главное image + список/лента остальных + выбор image.
- Backend уже задает порядок; не пересортировывать.
- Local selected media хранить signal.
- Не использовать carousel dependency.

Variant media

- Когда выбран ProductVariant и у него есть media - показать его media в соответствующем месте.
- Не придумывать скрытый fallback между product/variant media.
- General gallery отображает Product.media.
- Selected variant media отображает Variant.media.
- Автоматическая замена всей gallery variant images - отдельное UX-решение.

SSR

- Не использовать browser-only API без необходимости.
- Обычные image URLs должны работать при SSR/hydration.

Не делать

- upload UI.
- image editing.
- thumbnail generation.
- S3 logic.
- Google Drive logic.
- compression.
- external gallery library.
- zoom/lightbox без отдельной задачи.

Проверка

- Product с primary image.
- Product без primary image.
- Detail gallery.
- Смена selected image.
- Variant with media.
- URLs через frontend proxy.
- Тесты/build без отдельной команды не запускать.

Catalog Angular - 6. Stock

Зависимость: Catalog Backend - 6 и Catalog Angular - 2 завершены.

Цель: показывать реальное availability ProductVariant через in_stock.

Types

- ProductVariant.in_stock:boolean.
- default_variant.in_stock:boolean.
- Не добавлять quantity/reserved, если public API их не возвращает.

Product cards

- default_variant != null и in_stock=true -> 'В наличии'.
- default_variant != null и in_stock=false -> 'Нет в наличии'.
- default_variant=null -> не считать Product автоматически available/unavailable.

Product detail

- VariantSelector показывает availability каждого variant.
- Out-of-stock variant остается видимым.
- Визуально показать 'Нет в наличии'.
- Запретить выбирать его как доступный к покупке.
- Пока cart нет - не добавлять Add to cart.
- Если default variant out of stock - показать цену/характеристики и 'Нет в наличии'.
- Не выбирать первый in-stock variant автоматически.

UI

- Не использовать цвет как единственный индикатор availability; текст обязателен.

Не делать

- quantity-reserved calculation.
- Stock endpoint requests.
- reservation.
- cart.
- backorder.
- notification о поступлении.
- local stock state.

Проверка

- in_stock=true.
- in_stock=false.
- default out of stock.
- несколько variants с разным availability.
- Тесты/build без отдельной команды не запускать.

Catalog Angular - 7. ProductCompatibility

Зависимость: Catalog Backend - 7 и Catalog Angular - 2 завершены; если используются images - также media-блок.

Цель: добавить на Product detail связанные товары по ProductCompatibility.

Types

- compatible_with: ProductCompatibilitySummary[].
- compatible_items: ProductCompatibilitySummary[].
- Использовать только поля real backend contract: id, name, slug, brand|null, primary_image|null если есть.
- Не дополнять objects дополнительными запросами по одному Product.

Product detail

- Блок 'Совместимо с' <- compatible_with.
- Блок 'Подходящие аксессуары' <- compatible_items.
- Пустой массив -> блок не отображать.

Compatibility card

- Компактный reusable component.
- Image, если API предоставляет.
- Product name.
- Brand, если есть.
- Ссылка на /catalog/products/:slug.
- Не использовать mock image при null.
- Не дублировать полный ProductCard, если response не содержит всех его данных.

Navigation

- Клик открывает Product detail.
- Проверить переходы в обе стороны.
- Не создавать обратную compatibility на frontend.

Не делать

- fuzzy compatibility.
- поиск по имени модели.
- SKU compatibility.
- CompatibilityType UI.
- 'похожие товары'.
- recommendations engine.
- дополнительные API-запросы для каждой карточки.

Проверка

На примере Silicone Case -> iPhone 16:

- У Case 'Совместимо с' содержит iPhone 16.
- У iPhone 16 'Подходящие аксессуары' содержит Case.
- Переходы по ссылкам.
- Пустые блоки не отображаются.
- Тесты/build без отдельной команды не запускать.

6. Общие контрольные правила

- Не выполнять git add/commit/push/pull/fetch/merge/rebase/checkout/switch без явной команды.
- Не устанавливать pip/npm dependencies без подтверждения.
- Не запускать migrate/makemigrations без разрешения, кроме случаев, где конкретное ТЗ прямо разрешает makemigrations.
- Не запускать Docker state-changing команды без подтверждения.
- Тесты не создавать, не изменять и не запускать без прямого требования.
- Backend API URLs только под /api/; Angular использует относительные /api/... paths.
- Django views остаются тонкими; не вводить services/selectors/repositories для простого CRUD.
- Angular: standalone, OnPush, signals where appropriate, reactive forms, @if/@for, strict TS/templates, Tailwind.
- Изображения не хранить бинарно в PostgreSQL. Production direction - S3-compatible storage, но S3 config не добавлять без отдельной задачи.
- После каждой задачи кратко перечислить changed files, что сделано, что проверено и что не проверено.

7. Границы текущего этапа

- В этом документе описан только catalog implementation roadmap.
- Cart, checkout, orders, addresses, payments, shipment/delivery, discounts/coupons проектируются отдельными этапами.
- Не создавать эти сущности заранее.
- История будущих заказов должна использовать snapshots, а CartItem/OrderItem должны ссылаться на ProductVariant.
- Payment/Shipment не привязывать к конкретному provider до выбора реального сервиса.